

```

</CsOptions>
<CsInstruments>
sr = 16000
kamps = 160
nchnls = 2
glsf sfload "/usr/share/soundfonts/PCLite.sf2"
sfassign 0, glsf

instr 1
ivel = p4
inotenum = p5
ipreindex = p6
iamp = p7
a1, a2 sfplay ivel, inotenum, iamp, 1, ipreindex, 0
outs a1, a2
endin
</CsInstruments>
<CsScore>
</CsScore>
</CsoundSynthesizer>

```

The above file structure is the same as what was discussed in the last article. The SoundFonts need to be loaded and presets (instruments) assigned. An orchestra instrument can play the SoundFont notes using the *sfplay* opcode. You can find out more details at www.csound.com/manual/html/sfplay.html.

The *ivel* parameter has a value between 0 and 127, and is the velocity or the vigour with which a musician plays the note—this translates into volume. You can use the *iamp* parameter as a factor to further adjust the volume.

The frequency is determined by *inotenum*, which also ranges between 0 and 127. The MIDI notes between 48 and 72 correspond to frequencies between 130.81Hz and 523.25Hz (www.phys.unsw.edu.au/jw/notes.html). This range shows the most noticeable difference between the various presets.

You may wish to experiment with various preset indexes, for example, 0 is a grand piano, 24 is a guitar, 40 is a violin, 104 is a sitar, and 117 is a drum. The index is displayed when you run *csound* for the above *csd* file (on Fedora, but not on Ubuntu).

In the case of Ubuntu, you will need to replace the *sfload* command in the *sf2.csd* file by using the following code:

```
glsf sfload "/usr/share/sounds/sf2/FluidR3_GM.sf2"
```

A better definition of an instrument to play SoundFonts will be more complex.

The Python code to explore the differences

Write the following code in *play_sf2.py*:

```
# Import the Csound API extension module.
import csnd
```

```
import sys
```

```
def add_score(csound, inst, dur, amp):
    time = 0
    vel = 127
    for note in range(48, 72):
        csound.addNote(1, time, dur, vel, note, inst, amp)
        time += dur
    csound.addScoreLine('e 2')

# Command line parameters:
# Instrument Index, note duration, amplification
inst = int(sys.argv[1]) if len(sys.argv) > 1 else 0
note_duration = float(sys.argv[2]) if len(sys.argv) > 2 else 0.5
amp = float(sys.argv[3]) if len(sys.argv) > 3 else 1.0
print 'Instrument, note duration, amplification:', inst, note_duration, amp
# Create an instance of Csound.
csound = csnd.CppSound()
csound.setPythonMessageCallback()
csound.load('sf2.csd')
csound.setCommand('csound -dm0 -T -odac -rtaudio=alsa temp.orc
temp.sco')
add_score(csound, inst, note_duration, amp)
csound.exportForPerformance()
csound.perform()
```

The code is very similar to what we explored last month. The *add_score* method inserts notes to play on the first orchestra instrument.

You may pass command line parameters to specify the preset index, the note duration and the amplification. By default, the program will play a piano with each note for half a second and an amplification factor of 1.

The *csound* command in the code plays the notes directly and does not create an intermediary wave file.

Try the following options to compare the notes of a piano being played for a short and for a longer duration.

```
$ python play_sf2.py 0 0.3 2
```

```
$ python play_sf2.py 0 2 2
```

Now, compare a sitar with a violin:

```
$ python play_sf2.py 104 2 2
```

```
$ python play_sf2.py 40 2 2
```

As you can imagine, you can easily spend a few hours studying the differences.

A one-button music player

So far, you have been writing a program and playing the notes. Can you program *csound* to play notes interactively, for example, by pressing buttons or keys? The answer is, of course, yes!

The following command line option for *csound* will help: